

The slide features a light gray background with several decorative elements: a large white circle in the top-left corner, a solid purple circle in the top-left, a purple-to-white gradient bar in the top-right, a purple-to-white gradient bar in the bottom-left, a small purple circle in the bottom-right, and a large white circle in the bottom-right. The main title is centered in the upper half of the slide.

PROG IV: **Aula de abertura**

Prof^a. Tainá Isabela



Apresentação

Acadêmica:

- Técnico em Informática - IFMT
- Bacharel em Ciências da Computação - UFMT
- Pós em BI, Big Data e Inteligência Artificial

Profissional:

- Desenvolvedora Back-end desde 2021;
- 

Plano de Ensino

Ementa

- Arquiteturas e tecnologias computacionais para desenvolvimento de frontend e backend para ambientes Web e/ou mobile.
- Linguagens de programação de desenvolvimento de frontend e backend para ambiente Web e/ou mobile.
- Aplicações.

Conteúdo Programático

- 1. Fundamentos de Aplicações Web
 - 1.1 Arquitetura cliente-servidor
 - 1.2 Funcionamento da Web (HTTP/HTTPS)
 - 1.3 Estrutura de aplicações modernas (Frontend + Backend + Banco)
 - 1.4 Introdução a APIs e padrão REST
- 2. Frontend Web (Base do Cliente)
 - 2.1 HTML (estrutura de páginas)
 - 2.2 CSS (estilização e layout)
 - 2.3 JavaScript (manipulação do DOM)
 - 2.4 Responsividade (mobile-first) 2.5 Acessibilidade (boas práticas)
 - 2.6 Consumo de APIs externas com JavaScript (fetch/axios)

Conteúdo Programático

- 3. Backend com NestJS (Base do Mercado)
 - 3.1 Introdução ao Node.js e ao NestJS
 - 3.2 Estrutura de projeto (modules, controllers, services)
 - 3.3 Criação de rotas (CRUD)
 - 3.4 Padrão REST (boas práticas de endpoints)
 - 3.5 Validação de dados (DTOs)
- 4. Banco de Dados e Persistência
 - 4.1 Modelagem de dados
 - 4.2 Banco relacional com MySQL
 - 4.3 Integração com backend
 - 4.4 ORM (ex: TypeORM)
 - 4.5 Operações CRUD no banco

Conteúdo Programático

- 5. Integração Completa (Full Stack)
 - 5.1 Integração frontend + backend
 - 5.2 Requisições HTTP (GET, POST, PUT, DELETE)
 - 5.3 Consumo de APIs externas no backend
 - 5.4 Tratamento de erros
- 6. Autenticação e Segurança
 - 6.1 Conceito de autenticação vs autorização
 - 6.2 Implementação com JWT
 - 6.3 Proteção de rotas
 - 6.4 Noções de segurança (hash de senha, boas práticas)

Conteúdo Programático

- 7. Qualidade e Boas Práticas
 - 7.1 Versionamento com Git
 - 7.2 Organização de código (boas práticas)
 - 7.3 Testes unitários (backend)
 - 7.4 Padronização de projetos
- 8. Projeto Prático (Baseado no Mercado)
 - 8.1 Definição do sistema (ex: API + frontend simples)
 - 8.2 Desenvolvimento incremental (CRUD completo)
 - 8.3 Integração com API externa
 - 8.4 Implementação de autenticação (JWT)
 - 8.5 Testes e versionamento 8.6 Deploy conceitual / apresentação final

Avaliação

Questionários e Exercícios (Peso 3,0)

Atividades ao longo do curso para fixação do conteúdo.

Trabalho Prático (Peso 3,0)

Trabalhos (T1 e T2) envolvendo atividades aplicadas, estudos dirigidos, projetos ou produções correlatas aos conteúdos ministrados;

Prova Final (Peso 4,0)

Provas (P1 e P2) destinadas à verificação do domínio dos conteúdos teóricos e práticos abordados na disciplina;

Bibliografia

- SILVA, Osmar J. JavaScript: guia prático do Webmaster. São Paulo: Érica, 2000. 264 p.
- Lubbers, Peter; Albers, Brian, Programação Profissional Em Html 5. Alta Books, 2013.
- SILVA, Osmar J. DHTML: dynamic HTML: estilos e conteúdo dinâmico. São Paulo: Érica, 2001. 252

The slide features a light gray background with several decorative elements: a large white circle in the top-left corner, a solid purple circle in the top-left area, a purple horizontal bar in the top-right corner, a small purple circle in the bottom-right area, a large white circle in the bottom-right corner, and a purple horizontal bar in the bottom-left corner.

Prog IV: **Fundamentos de** **Aplicações Web**

Prof^a. Tainá Isabela

Arquitetura Cliente-Servidor

“É uma arquitetura onde o processamento da informação é dividido em módulos ou processos distintos. Um processo é responsável pela manutenção da informação (Servidor), enquanto que outro é responsável pela obtenção dos dados (Cliente)”.

A arquitetura cliente-servidor é um modelo fundamental da computação moderna, especialmente no contexto da Web. Ela define como sistemas são organizados e como diferentes partes de uma aplicação se comunicam.

Arquitetura Cliente-Servidor

“É uma abordagem da computação que separa os processos em plataformas independentes que interagem, permitindo que os recursos sejam compartilhados enquanto se obtém o máximo de benefício de cada dispositivo diferente, ou seja, Cliente/Servidor é um modelo lógico.”

Nesse modelo, existem dois papéis principais:

- Cliente: responsável por fazer requisições;
- Servidor: responsável por processar requisições e retornar respostas;

Arquitetura Cliente-Servidor

Entre suas características principais se incluem:

- Separação de responsabilidades;
- Comunicação via rede;
- Possibilidade de múltiplos clientes para um único servidor;
- Escalabilidade (um servidor pode atender milhares de usuários);



Vantagens

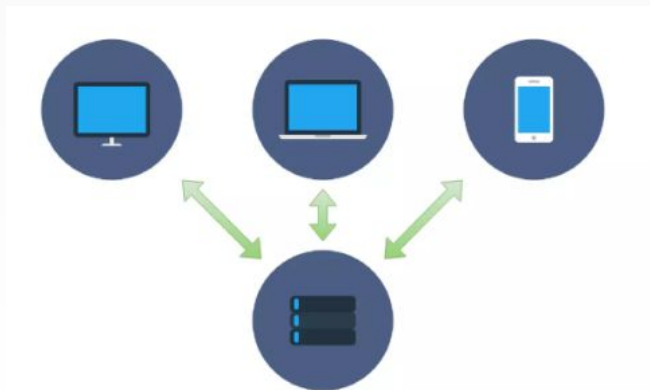
- Organização modular do sistema
- Facilidade de manutenção
- Melhor controle de dados e segurança

Desvantagens

- Dependência do servidor
- Possíveis gargalos de desempenho
- Necessidade de conexão com a rede

Arquitetura Cliente-Servidor

Estrutura Básica: Temos servidores para arquivos, banco de dados, e outras funções, tais como: servidores de impressão, servidores web, etc.



Modelos de Arquitetura Cliente-Servidor

Arquitetura C/S Simples:

Nesta arquitetura, o servidor não pode iniciar nada. O Servidor somente executa as requisições do Cliente. Existe uma clara função de diferenciação. Pode-se estabelecer que o Cliente é o mestre e o Servidor o vassalo.

Modelos de Arquitetura Cliente-Servidor

Arquitetura C/S Simples:

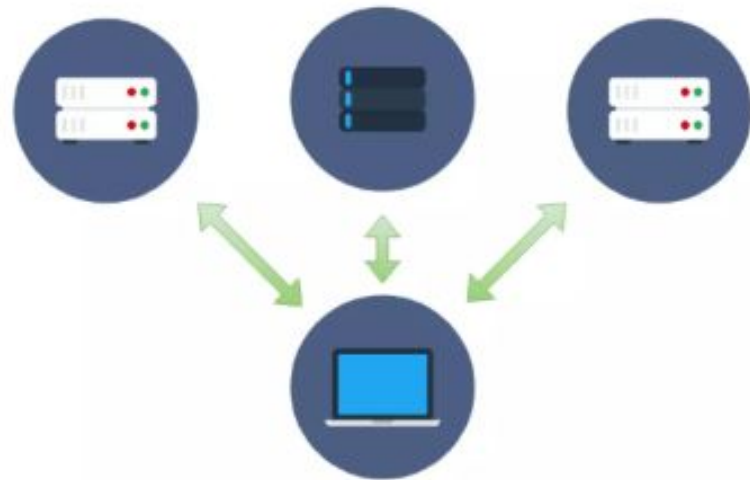
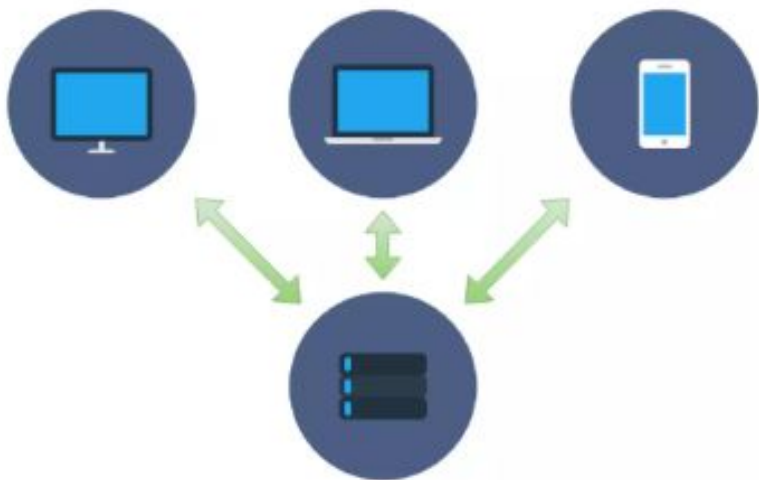
Nesta arquitetura, o servidor não pode iniciar nada. O Servidor somente executa as requisições do Cliente. Existe uma clara função de diferenciação. Pode-se estabelecer que o Cliente é o mestre e o Servidor o vassalo.

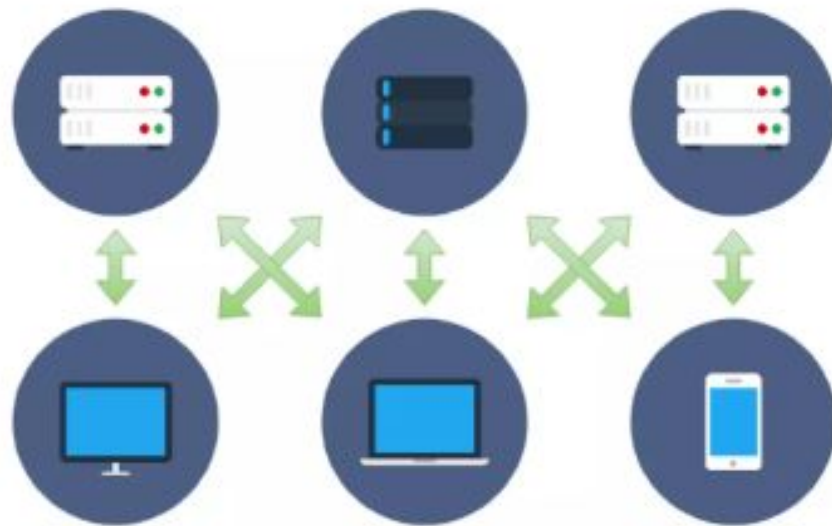


Modelos de Arquitetura Cliente-Servidor

Arquitetura C/S em Dois Níveis:

A configuração usual Cliente/Servidor encontrada na maioria das empresas, é aquela que existem vários clientes requisitando serviços a um único Servidor. Esta arquitetura se caracteriza como sendo centrada no Servidor. Porém na visão do usuário, ele imagina que existem vários Servidores conectados a somente um cliente, ou seja, centrado no Cliente. Entretanto, com as várias ligações possíveis, existe na realidade uma mistura de Clientes e Servidores.

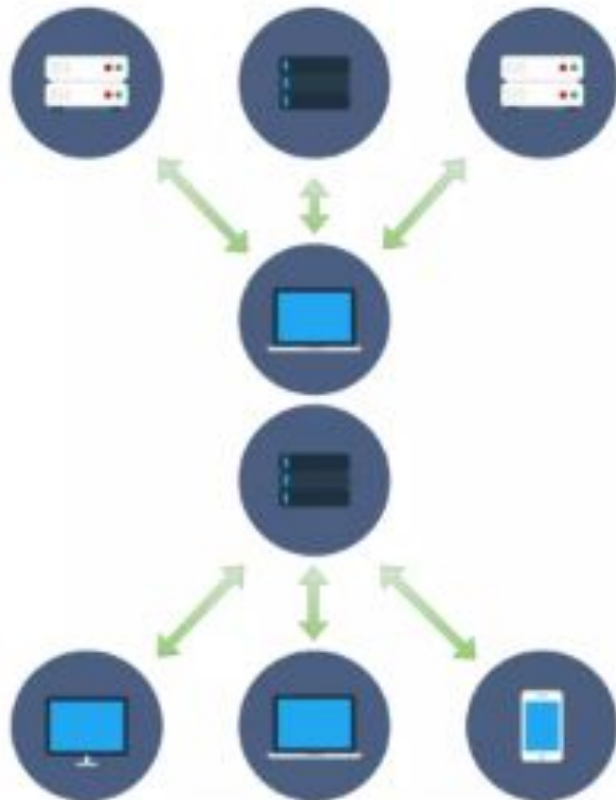




Modelos de Arquitetura Cliente-Servidor

Arquitetura C/S Multinível:

Nesta arquitetura, permiti-se que uma aplicação possa assumir tanto o perfil do Cliente como o do servidor, em vários graus. Em outras palavras, uma aplicação em alguma plataforma será um servidor para alguns Clientes e, concorrentemente, um cliente para alguns Servidores.



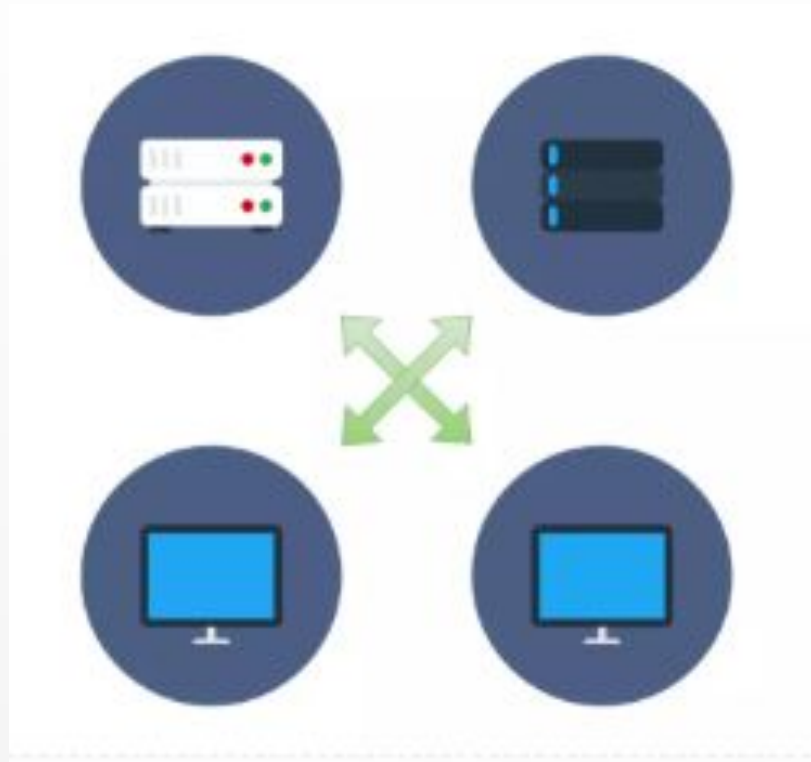
.....

Modelos de Arquitetura

Cliente-Servidor

Arquitetura C/S Par-Par:

Esta arquitetura pode ser vista como o caso mais geral da arquitetura Cliente/Servidor. Cada um dos nodos desta arquitetura assume tanto o papel de Cliente quanto de Servidor. Na verdade, torna-se pouco funcional lidar com quem é o Cliente ou o Servidor. É o caso onde o processo interage com outros processos em uma base pareada, não existindo nenhum Mestre ou Vassalo: Qualquer estação de trabalho pode iniciar um processamento, caso possua uma interface de comunicação entre o usuário e o processo Cliente.



Funcionamento da Web (HTTP/HTTPS)

A Web funciona com base em protocolos de comunicação, sendo o principal o HTTP.

HTTP (**HyperText Transfer Protocol**) é o protocolo responsável pela comunicação entre cliente e servidor na Web.

Ele funciona no modelo requisição → resposta.

Estrutura de uma requisição HTTP

Uma requisição contém:

- Método (GET, POST, PUT, DELETE)
- URL (endereço do recurso)
- Cabeçalhos (headers)
- Corpo (body, opcional)

Exemplo:

```
GET /index.html HTTP/1.1  
Host: www.exemplo.com
```

Protocolo HTTP

Requisição:

Na requisição, é onde será feito o envio e/ou a requisição de dados para o servidor. Por exemplo, enviar o login e a senha de um usuário e o servidor retornará através da Resposta se aquele usuário é válido ou não.

Protocolo HTTP

Resposta:

A resposta do servidor servirá apenas para orientar a camada de cliente (frontend). Sempre que uma requisição é feita, a resposta é separada em duas partes, uma é composta pelo Http Status Code, que são códigos para definir o sucesso ou a falha de uma operação e a outra pelo corpo da resposta, que contém os arquivos e dados solicitados. Por exemplo, caso você esteja executando um GET para uma lista de usuários, o corpo da requisição retornará essa lista em formato JSON.

Protocolo HTTP

Os códigos de estado HTTP são desenvolvidos de acordo com os padrões da Internet definidos pela Internet Engineering Task Force (IETF). Estão classificadas em cinco categorias diferentes, conforme segue.:

- 1xx série – Mensagem Informativa
- 2xx – Mensagem de Sucesso
- 3xx – Mensagem de Redirecionamento
- 4xx – Mensagens de Erro Relacionadas com o Cliente
- 5xx – Mensagens de Erro Relacionadas com o Servidor

Protocolo HTTP

Os códigos de estado HTTP são desenvolvidos de acordo com os padrões da Internet definidos pela Internet Engineering Task Force (IETF). Estão classificadas em cinco categorias diferentes, conforme segue.:

- 1xx série – Mensagem Informativa
- 2xx – Mensagem de Sucesso
- 3xx – Mensagem de Redirecionamento
- 4xx – Mensagens de Erro Relacionadas com o Cliente
- 5xx – Mensagens de Erro Relacionadas com o Servidor

Método GET

Esse método é o mais utilizado, tem a função de **solicitar ou requisitar algum recurso ao servidor**.

É geralmente usado para o **retorno de informações** sobre uma entidade especificamente ou uma lista delas.

Método POST

Método usado para enviar informações ao servidor, é utilizado de forma mais popular em **cadastros de dados** retirados de um formulário, e também quando enviamos um arquivo para o servidor: seja isso uma foto de perfil, um post no Instagram, tudo isso é usado o método POST.

Método PUT

O PUT é responsável por **editar** arquivos e informações já existentes. Esse método usa-se juntamente com um parâmetro na URL da requisição, que ajudará a **identificar** através de uma chave única (**id**) qual elemento ele está referenciando.

Método PATCH

O método PATCH é usado para aplicar **modificações parciais** a uma entidade existente no servidor. Em vez de enviar todos os dados da entidade como no PUT, o cliente envia apenas os dados que precisam ser modificados ou atualizados. Isso torna o PATCH útil quando apenas **partes específicas** de uma entidade precisam ser alteradas, sem afetar o restante dos dados.

pet Everything about your Pets

[Find out more](#) ^

PUT

/pet Update an existing pet



POST

/pet Add a new pet to the store



PATCH

/pet/change-name Change pet name



Método DELETE

Esse método é bem autoexplicativo, **deleta** arquivos ou informações presentes no banco de dados. Igual o PUT, é recomendável que utilize-o usando um parâmetro na URL da requisição, para que evite problemas de deletar dois arquivos/informações ao mesmo tempo.

Método HEAD

Esse método é muito similar com o GET, a diferença é que esse retorna apenas o cabeçalho da resposta, enquanto o GET retorna tanto o cabeçalho quanto o corpo da resposta.

Método OPTIONS

É responsável por retornar as informações referentes ao servidor. Esse método é muitíssimo importante para ferramentas de deploy como o Heroku, pois ele pega todas as informações necessárias para o deploy de forma automática por meio desse método, o que facilita a vida do programador.

Método TRACE

Funciona bem mais como uma ferramenta de debug para as API 's. Ele reenvia a última requisição para o servidor e verifica se houve alguma interferência de servidores de terceiros.

HTTPS (HTTP Seguro)

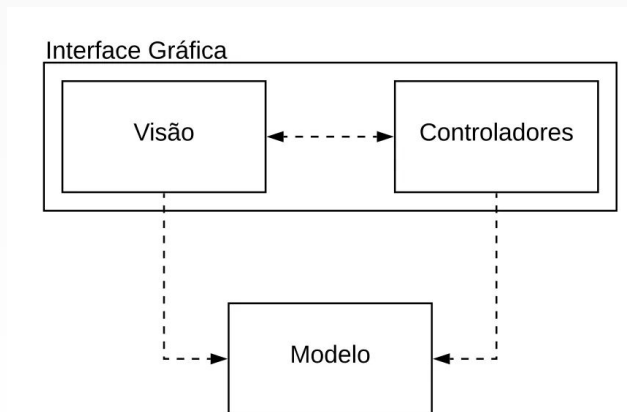
HTTPS é a versão segura do HTTP.

- Utiliza criptografia (SSL/TLS);
- Protege dados sensíveis (senhas, cartões);
- Garante:
 - Confidencialidade;
 - Integridade;
 - Autenticidade;

Estrutura de uma Aplicação Web

Uma aplicação web moderna é dividida em três camadas principais:

- Frontend;
- Backend;
- Banco de dados;



Application Layers

User Interface

Business Logic

Data Access



Frontend (Camada de Apresentação)

O frontend é a parte visual da aplicação, com a qual o usuário interage.

Tecnologias principais:

- HTML → estrutura
- CSS → estilo
- JavaScript → comportamento

Responsabilidades:

- Interface do usuário (UI)
- Experiência do usuário (UX)
- Comunicação com o backend

Seja Bemvindo ao nosso site!

Feliz a Nação cujo Deus é o Senhor!

ÔNIBUS - CAMINHÕES - AÉREO - NÁUTICO - AGRÍCOLA
PARTICULAR - ESPECIAL - GELADEIRAS AUTOMOTIVAS



[Criação de Sites](#)



Papelaria Potência Design
Fone: 11 3

Entrar

Backend (Camada de Negócio)

O backend é responsável pela lógica da aplicação.

O que ele faz:

- Processa requisições
- Aplica regras de negócio
- Comunica com o banco de dados

Exemplos de tecnologias:

- Python (Flask, Django)
- Node.js
- Java (Spring)
- PHP

Insomnia 3.0

POST {{ base_url }}/send

Send

200 OK

TIME 538 ms

SIZE 358 B

Production

Cookies

Filter

Send Email

POST With Data

POST Segment

POST A/B Test

ESP Accounts

GET Create

Drip Campaigns

Customers

Conversions

Templates

JSON

Auth

Params

Headers (1)

```
1 {
2   "template": "{{ template_id }}",
3   "recipient": {
4     "address": "julia@fakename.xyz",
5     "name": "Julia Rylie"
6   },
7   "sender": {
8     "address": "support@insomnia.rest",
9     "name": "Gregory Schier"
10  },
11  "template_data": {
12    "name": "{{ name | title }}",
13    "age": 26
14  }
15 }
```

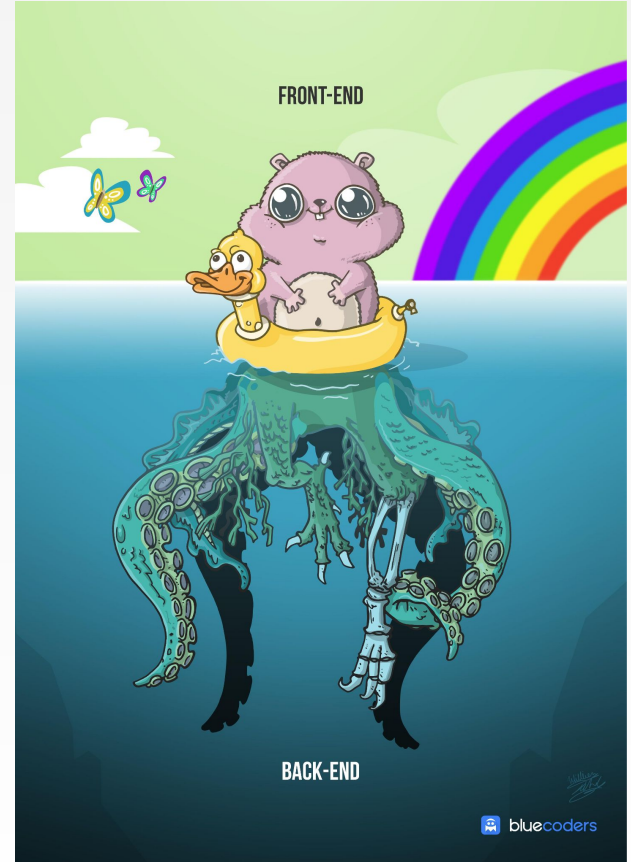
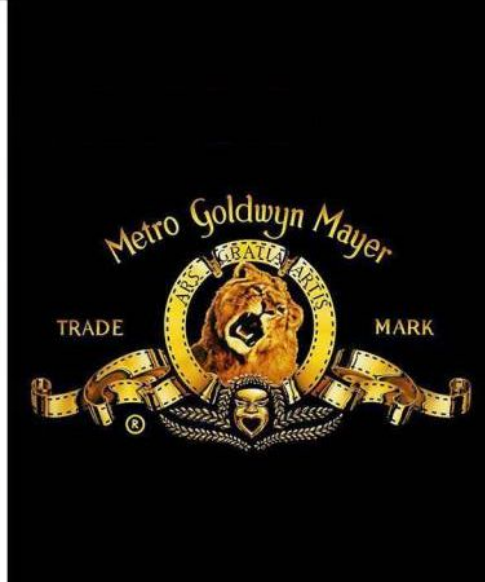
Source

Cookies

Headers (7)

```
1 {
2   "success": true,
3   "receipt_id":
4   "log_40e7828498f2aaf4f2be7e47f0bc3aeb",
5   "email": {
6     "name": "Invite",
7     "version_name": "New Version",
8     "locale": "en-US"
9   },
10  "status": "OK"
11 }
```

Programming: BACKEND vs FRONTEND



Banco de Dados (Camada de Dados)

É onde os dados são armazenados de forma persistente.

Tipos:

- Relacionais (SQL);
 - MySQL, PostgreSQL;
- Não relacionais (NoSQL);
 - MongoDB;

Funções:

- Armazenar dados;
- Consultar informações;
- Garantir integridade;

MySQL Workbench

teste x

File Edit View Query Database Server Tools Scripting Help

Navigator

SQL File 11" x

Limit to 500 rows

SCHMAS

Filter objects

sys

Administration Schemas

Information

No object selected

```
-- Script para criação das tabelas

CREATE DATABASE loja_cliente;
USE loja_cliente;

CREATE TABLE LOJA (
  loja_id INTEGER NOT NULL,
  nome VARCHAR(100) NOT NULL,
  CONSTRAINT LOJA_PK PRIMARY KEY(loja_id)
) ENGINE = InnoDB;

CREATE TABLE CLIENTE (
  id INTEGER NOT NULL AUTO_INCREMENT,
  loja_id INTEGER NOT NULL,
  nome VARCHAR(100) NOT NULL,
  cpf NUMERIC(11) NOT NULL,
  rua VARCHAR(100) NOT NULL,
  cidade VARCHAR(100) NOT NULL DEFAULT 'Brasilia',
  estado VARCHAR(2) NOT NULL DEFAULT 'DF',
  credito_limite DECIMAL(7,2) NOT NULL,
  CONSTRAINT CLIENTE_PK PRIMARY KEY(cliente_id),
  CONSTRAINT CLIENTE_LOJA_FK FOREIGN KEY(loja_id) REFERENCES LOJA(loja_id),
  CONSTRAINT CLIENTE_CK CHECK(credito_limite >= 0),
  CONSTRAINT CLIENTE_UQ UNIQUE(cpf)
) ENGINE = InnoDB AUTO_INCREMENT = 1;
```

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	15:18:18	CREATE DATABASE loja_cliente	1 row(s) affected	0.016 sec
2	15:18:18	USE loja_cliente	0 row(s) affected	0.000 sec
3	15:18:18	CREATE TABLE LOJA (loja_id INTEGER NOT NULL, nome VARCHAR(100)	0 row(s) affected	0.031 sec
4	15:18:18	CREATE TABLE CLIENTE (id INTEGER NOT NULL AUTO_INCREMENT, ...	Error Code: 1072. Key column 'cliente_id' doesn't exist in table	0.000 sec

Frontend



Navegador (Cliente)



Backend



Servidor

Banco de Dados

Fluxo Web Simples

Fluxo completo de uma aplicação

1. Usuário interage com o frontend;
2. Frontend envia requisição ao backend;
3. Backend processa e acessa o banco de dados;
4. Backend retorna resposta;
5. Frontend exibe o resultado;

Introdução a APIs

API (Application Programming Interface) é um conjunto de regras que permite que sistemas diferentes se comuniquem.

Exemplo simples:

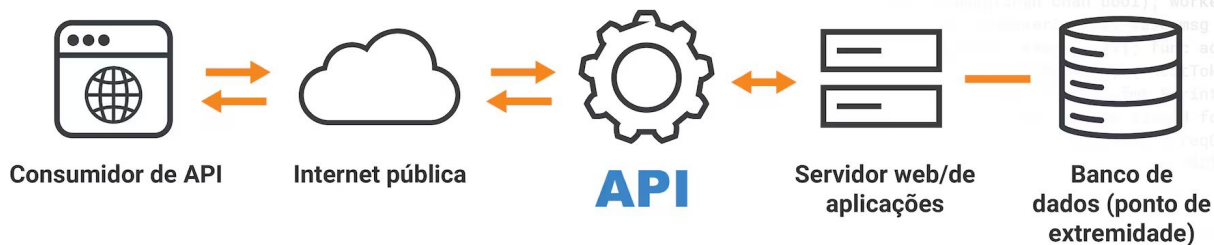
Um app de clima acessa dados de um servidor externo via API

Introdução a APIs

- Integração entre sistemas;
- Reutilização de serviços;
- Desenvolvimento mais rápido;
- Separação entre frontend e backend;

Tipos de APIs

- APIs públicas;
- APIs privadas;
- APIs internas (microserviços);



Conceito de REST

REST (**Representational State Transfer**) é um estilo arquitetural para construção de APIs.

Princípios do REST

- Comunicação via HTTP;
- Uso de URLs para recursos;
- Stateless (sem estado);
- Uso de métodos HTTP corretamente;

Conceito de REST

No REST, tudo é tratado como um recurso.

Exemplo:

/usuarios

/produtos

/pedidos

Exemplos de operações REST

GET /usuarios → listar usuários

GET /usuarios/1 → buscar usuário específico

POST /usuarios → criar usuário

PUT /usuarios/1 → atualizar usuário

DELETE /usuarios/1 → deletar usuário

GET

/produtos/123

PUT

/produtos/123

DELETE

/produtos/123

POST

/produtos

URL que vamos acessar

```
http://devmedianotesapi.azurewebsites.net/api/Notes/2505
```

Retorno da requisição (JSON)

```
{  
  "Id": 2505,  
  "Title": "título",  
  "Body": "Curso"  
}
```

Questionário

